# Leitura — Um serviço em Go, gRPC e implantação (Sprint 2)

Guia de apoio para as duas aulas da Sprint 2: introdução a Go, com os exemplos do Curso básico de Go, e o domínio do serviço Go (21/10, Passo 13); o contrato em Protocol Buffers, o servidor gRPC, a integração com a API, os containers e a implantação (26/10, passos 14 a 18). Na Sprint 1, a API de Tarefas ganhou banco, testes, validação e camadas. Nesta, ela passa a conversar com um segundo serviço, escrito em Go, e vai para o ar.

Exemplo de referência: `exemplos/avisos-go/` (com o `PASSOS.md` dos passos 13 a 18), `exemplos/protos/`, `exemplos/ktor-tarefas/`, `exemplos/quarkus-tarefas/`, `exemplos/compose.yaml` e `exemplos/implantacao/`. Os slides são `slides/web2-slides-07.md` e `slides/web2-slides-08.md`. No branch `aula`, as tags `passo-13` a `passo-18` guardam o estado de cada passo.

Versões: Go 1.26, grpc-go 1.84.0, protobuf-go 1.36.12, Buf CLI 1.71, grpc-java 1.85.0, grpc-kotlin 1.5.0, protobuf 4.36.2, plugin Gradle `com.google.protobuf` 0.10.0 e a extensão `quarkus-grpc` do Quarkus 3.28.2. Tudo o que está descrito foi executado: os testes (3 + 3 em Go, 16 no Ktor, 13 no Quarkus), o `grpcurl` contra o servidor e o `docker compose` com os quatro serviços saudáveis.

Capítulos:


1. A Sprint 2 e o serviço de avisos
2. Go para quem vem de Kotlin e Java
3. O domínio em Go: interfaces, concorrência, `context` e erros
4. O contrato: Protocol Buffers e Buf
5. O servidor gRPC
6. A API chama o Go
7. Containers e `docker compose`
8. Implantação
9. Exercícios e dúvidas frequentes


## 1. A Sprint 2 e o serviço de avisos

O que a Sprint 2 cobra:

| Critério | Peso | Capítulos |
|---|---|---|
| Microsserviço Go | 25% | 1, 2 e 3 |
| Contrato Protobuf e integração gRPC | 25% | 4, 5 e 6 |

| Critério | Peso | Capítulos |
|---|---|---|
| Sistema no ar | 25% | 8 |
| Ambiente e testes | 15% | 6 e 7 (e os testes que saíram da Sprint 1) |
| Documentação | 10% | 8 |

No exemplo, a responsabilidade do serviço Go é avisar que uma tarefa foi criada, por três canais (e-mail, push e chat, simulados). Quando uma tarefa é criada, a API chama o serviço por gRPC; o serviço avisa os três canais ao mesmo tempo e responde quais entregaram.

A rubrica pede uma responsabilidade **justificada**. A justificativa do exemplo:

- **É trabalho de E/S que espera por terceiros.** Cada canal espera um servidor externo. Com uma goroutine por canal, o tempo total é o do canal mais lento, e não a soma.
- **A falha fica isolada.** Se o serviço cair, a API continua criando tarefas.
- **Tem ritmo próprio.** Um canal novo muda o serviço, sem republicar a API.

Separar em serviços tem custo: mais um processo para implantar, monitorar e manter compatível. Um serviço que divide o banco com a API, ou que só repassa chamadas, costuma custar mais do que rende. A ADR do grupo deve dizer por que o serviço existe e o que aconteceria sem ele.

## 2. Go para quem vem de Kotlin e Java

| Assunto | Kotlin e Java | Go |
|---|---|---|
| Unidade de código | classe e pacote | pacote: uma pasta, com vários arquivos; o módulo é o `go.mod` |
| Visibilidade | `private`, `public` | inicial maiúscula é exportada (`Avisar`); minúscula, não (`canais`) |
| Objetos | classes, herança | `struct` com métodos; composição em vez de herança |
| Interfaces | `implements`, explícito | satisfeitas implicitamente: quem tem os métodos implementa |
| Erros | exceções | valores: a função devolve `(T, error)`, e quem chama confere |
| Concorrência | threads, corrotinas | goroutines (`go f()`) e canais |
| Nulo | `null` | cada tipo tem um valor zero; `nil` para ponteiros, interfaces, fatias e mapas |
| Formatação | configurável | uma só, a do `gofmt` |

Um método em Go se declara fora do tipo, com um receptor entre parênteses antes do nome:

```
func (av *Avisador) Avisar(ctx context.Context, aviso Aviso) ([]Entrega, error)
//    └ receptor: o "this"  └ prazo e cancelamento          └ dois resultados: valor e erro
```

O `*` indica que o receptor é um ponteiro: o método age sobre o mesmo `Avisador`, e não sobre uma cópia. Construtores não existem; o costume é uma função `NovoAvisador(...)` que devolve o valor montado.

### 2.1 Aprender Go com testes

A aula de 21/10 usa os exemplos do Curso básico de Go, que segue o livro *Aprenda Go com Testes*: cada assunto é construído a partir de um teste que falha. O repositório dos exemplos tem uma pasta por aula, e cada uma roda com `go test -v`:

| Pasta | Assunto | O que levar para o serviço Go |
|---|---|---|
| `2-hello-world` | ferramentas, tipos, testes com `t.Run` e `t.Helper`; funções como valores (*functional options*) | o teste ao lado do código, desde o primeiro passo |
| `3-metodos-interfaces` | `struct`, métodos e interfaces satisfeitas implicitamente | interfaces pequenas, declaradas por quem usa |
| `4-ponteiros-erros` | receptor por valor × por ponteiro; erros como valores, com tipo próprio | receptor ponteiro quando o método altera; erros com nome ou com tipo |
| `5-injecao-de-dependencia` | `io.Writer`, e o teste com `bytes.Buffer` | depender de uma interface para poder testar |

Dois pontos que costumam pegar quem vem de Java:

- **Receptor por valor recebe uma cópia.** Em `func (c Carteira) Depositar(q int)`, o depósito acontece na cópia, e o saldo da carteira original não muda. Com `func (c *Carteira)`, o método age sobre a original. A regra: ponteiro quando o método altera o estado.
- **Dois jeitos de reconhecer um erro.** Um erro com nome (`var ErrAvisoInvalido = errors.New(...)`) é conferido com `errors.Is`, quando basta saber qual erro foi. Um tipo de erro (`ErroSaldoInsuficiente`, com os campos `Solicitado` e `Disponivel`) é recuperado com `errors.As(err, &alvo)`, quando quem chama precisa dos dados.

Para quem está começando em Go, o caminho sugerido é o Curso básico de Go, em vídeo, que segue o *Aprenda Go com Testes*: cada assunto é construído a partir de um teste. Os capítulos de métodos e interfaces, ponteiros e erros, e injeção de dependência cobrem o que o serviço de avisos usa; os de concorrência, `select` e `context`, do mesmo livro, cobrem o capítulo 3 desta leitura.

📖 Ref. Curso básico de Go (vídeos): https://www.youtube.com/playlist?list=PLTr2C2xXIaIqO7Yr7FHohhgG_OgONpYIx

📖 Ref. Exercícios do curso: https://github.com/classrooms-fmarquesfilho/aprenda-go-com-testes

📖 Ref. Aprenda Go com Testes: https://larien.gitbook.io/aprenda-go-com-testes/

📖 Ref. A Tour of Go: https://go.dev/tour/

📖 Ref. Effective Go: https://go.dev/doc/effective_go

## 3. O domínio em Go: interfaces, concorrência, `context` e erros

### 3.1 Interfaces pequenas, do lado de quem usa

```go
type Canal interface {
    Nome() string
    Enviar(ctx context.Context, aviso Aviso) error
}
```

Qualquer tipo com esses dois métodos é um `Canal`, sem declarar nada. O `CanalSimulado` do exemplo é um; um canal de e-mail de verdade seria outro. O costume em Go é declarar a interface no pacote que a usa (aqui, o domínio), com poucos métodos. É o mesmo princípio da porta da Sprint 1 (`RepositorioDeTarefas`), com menos cerimônia.

### 3.2 Uma goroutine por canal

```go
entregas := make([]Entrega, len(av.canais))
var wg sync.WaitGroup
for i, canal := range av.canais {
    wg.Go(func() {
        entregas[i] = Entrega{Canal: canal.Nome(), Erro: canal.Enviar(ctx, aviso)}
    })
}
wg.Wait()
```

- `wg.Go(f)` dispara `f` numa goroutine e a registra no `WaitGroup`; `wg.Wait()` espera todas terminarem.
- Cada goroutine escreve só na sua posição da fatia. Como nenhuma posição é escrita por duas goroutines, não é preciso trava.
- Desde o Go 1.22, cada volta do `for` tem as suas próprias `i` e `canal`; a goroutine captura os valores daquela volta.

Com canais de 400, 50 e 150 ms, a chamada leva cerca de 400 ms. Em sequência, levaria 600.

📖 Ref. Go — sync.WaitGroup: https://pkg.go.dev/sync#WaitGroup

### 3.3 `context`: prazo e cancelamento

O `context.Context` carrega o prazo (deadline) e o sinal de cancelamento de uma operação. Por convenção, é o primeiro parâmetro de toda função que pode esperar.

```
func (c CanalSimulado) Enviar(ctx context.Context, aviso Aviso) error {
    select {
    case <-time.After(c.demora):          // terminou o trabalho
        return nil
    case <-ctx.Done():                    // o prazo venceu ou alguém cancelou
        return fmt.Errorf("canal %s: %w", c.nome, ctx.Err())
    }
}
```

O `select` espera o que acontecer primeiro. Quem chama decide quanto está disposto a esperar:

```
ctx, cancelar := context.WithTimeout(context.Background(), 100*time.Millisecond)
defer cancelar()
entregas, err := av.Avisar(ctx, aviso)
```

`defer` agenda a chamada para quando a função terminar, aconteça o que acontecer: é o lugar de liberar recursos.

📖 Ref. Go — context: https://pkg.go.dev/context

### 3.4 Erros como valores

```
var ErrAvisoInvalido = errors.New("aviso inválido")

return nil, fmt.Errorf("%w: tarefa %d, título %q", ErrAvisoInvalido, aviso.TarefaID,
        aviso.Titulo)

if errors.Is(err, avisos.ErrAvisoInvalido) { ... }
```

- Erros com nome (`ErrAvisoInvalido`) permitem que quem chama decida o que fazer, comparando com `errors.Is`, e não pelo texto da mensagem.
- `%w` embrulha um erro dentro de outro, acrescentando contexto sem perder a origem.
- Em vez de uma exceção que sai por um caminho escondido, o erro aparece na assinatura e é tratado ali, por quem chamou.

📖 Ref. Go — errors: https://pkg.go.dev/errors

### 3.5 Testes

Os testes de Go ficam ao lado do código (`avisos_test.go`), e rodam com `go test ./...`. O exemplo testa as três propriedades do domínio: os canais rodam ao mesmo tempo (a chamada leva menos de 550 ms), o prazo do `context` é respeitado (com 100 ms, só o push entrega) e o aviso inválido é recusado. O último usa um teste em tabela, o formato comum em Go.

## 4. O contrato: Protocol Buffers e Buf

### 4.1 O arquivo `.proto`

O contrato entre a API e o serviço Go é um arquivo de texto, versionado no repositório, a partir do qual cada linguagem gera o seu código:

```
syntax = "proto3";

package tarefas.avisos.v1;

option go_package = "avisos/gen/tarefas/avisos/v1;avisosv1";
option java_multiple_files = true;
option java_package = "br.ufrn.exemplo.tarefas.avisos.v1";

// Envia avisos sobre tarefas.
service AvisosService {
  // Avisa, em todos os canais ao mesmo tempo, que uma tarefa foi criada. ...
  rpc Avisar(AvisarRequest) returns (AvisarResponse);
}

// O pedido de aviso.
message AvisarRequest {
  // Identificador da tarefa criada. Precisa ser maior que zero.
  int64 tarefa_id = 1;
  // Título da tarefa. Não pode ficar em branco.
  string titulo = 2;
}
```

- `service` declara as operações; `message`, os dados.

- O número de cada campo (`= 1`, `= 2`) é o que identifica o campo no formato binário. Ele nunca muda e nunca é reaproveitado: é isso que permite acrescentar campos sem quebrar quem usa a versão anterior.
- A versão faz parte do pacote (`v1`). Uma mudança incompatível vira `v2`, ao lado.
- As opções dizem onde o código gerado fica em cada linguagem.

📖 Ref. Protocol Buffers — proto3: https://protobuf.dev/programming-guides/proto3/

### 4.2 Buf

O Buf confere e gera o contrato:

- `buf lint` aplica as regras do `buf.yaml`. O exemplo usa `STANDARD` (nomes, pacote com versão, sufixos `Request` e `Response`, serviço terminado em `Service`) e `COMMENTS` (todo elemento comentado, como pede a rubrica).
- `buf generate` gera os stubs conforme o `buf.gen.yaml`. O exemplo usa plugins remotos do Buf: nada de `protoc` instalado na máquina.

Os stubs Go ficam em `avisos-go/gen/` e entram no repositório. Os de Kotlin e Java não: o Gradle e o Quarkus os geram no build, a partir do mesmo `.proto` (capítulo 6).

📖 Ref. Buf — lint rules: https://buf.build/docs/lint/rules/

## 5. O servidor gRPC

### 5.1 O adaptador

O pacote `grpcapi` traduz entre o contrato gerado e o domínio. O domínio não importa nada de gRPC.

```
type Servidor struct {
    avisosv1.UnimplementedAvisosServiceServer
    avisador *avisos.Avisador
}
```

Embutir o tipo `Unimplemented...` é a recomendação do grpc-go: se o contrato ganhar uma operação nova, o servidor continua compilando e responde `UNIMPLEMENTED` para ela.

Os erros do domínio viram códigos gRPC:

| Situação | Código gRPC | Equivalente HTTP |
|---|---|---|
| entrada inválida (`ErrAvisoInvalido`) | `INVALID_ARGUMENT` | 400 |
| prazo vencido | `DEADLINE_EXCEEDED` | 504 |

| Situação | Código gRPC | Equivalente HTTP |
|---|---|---|
| serviço fora do ar | `UNAVAILABLE` | 503 |
| erro inesperado | `INTERNAL` | 500 |

📖 Ref. gRPC — Status codes: https://grpc.io/docs/guides/status-codes/

### 5.2 O prazo atravessa a rede

O cliente gRPC define um prazo, e ele chega ao servidor como o deadline do `ctx`. O servidor do exemplo encerra os envios 50 ms antes, a folga:

```
if prazo, ok := ctx.Deadline(); ok {
    var cancelar context.CancelFunc
    ctx, cancelar = context.WithDeadline(ctx, prazo.Add(-folga))
    defer cancelar()
}
```

Sem a folga, o servidor esperaria até o fim do prazo, e a resposta, com as entregas parciais, chegaria depois de o cliente ter desistido: ele veria só `DEADLINE_EXCEEDED`. Com a folga, um cliente com prazo de 300 ms recebe a lista: push e chat entregues, e-mail não.

📖 Ref. gRPC — Deadlines: https://grpc.io/docs/guides/deadlines/

### 5.3 Health check, reflection e encerramento gracioso

O `cmd/avisos/main.go` registra, além do serviço:

- o **health check** padrão do gRPC (`grpc.health.v1.Health`), que plataformas e o `docker compose` usam para saber se o serviço está de pé;
- o **reflection**, que permite a ferramentas como o `grpcurl` descobrir os serviços sem ter o `.proto`;
- o **encerramento gracioso**: ao receber SIGTERM (um `docker stop`, uma versão nova na plataforma), o servidor para de aceitar chamadas e termina as que estão em andamento.

```
grpcurl -plaintext localhost:50051 list
grpcurl -plaintext -d '{"tarefa_id": 7, "titulo": "Estudar gRPC"}' \
  localhost:50051 tarefas.avisos.v1.AvisosService/Avisar
grpcurl -plaintext localhost:50051 grpc.health.v1.Health/Check
```

📖 Ref. gRPC — Health checking: https://grpc.io/docs/guides/health-checking/

### 5.4 gRPC ou REST?

| | REST com JSON | gRPC com Protobuf |
|---|---|---|
| Contrato | OpenAPI | `.proto`, de onde o código é gerado |
| Formato | texto, legível | binário, compacto |
| Transporte | HTTP/1.1 ou 2 | HTTP/2 |
| Bom para | APIs públicas, navegador, apps | serviço falando com serviço, dentro do sistema |

No projeto, as duas convivem: REST na API que o cliente usa, gRPC entre a API e o serviço Go.

📖 Ref. gRPC — Introduction: https://grpc.io/docs/what-is-grpc/introduction/

## 6. A API chama o Go

### 6.1 Uma porta no domínio

```
fun interface Avisador {
    suspend fun avisar(tarefa: Tarefa): Boolean
}
```

O domínio não sabe que existe gRPC. A porta devolve `false` quando o aviso não foi possível, sem lançar exceção, porque para a API o aviso é acessório: a tarefa é o que importa. Na rota, depois de criar, `avisador.avisar(criada)`.

O teste de arquitetura da Sprint 1 passa a proibir também `io.grpc`, `com.google.protobuf` e o pacote gerado no domínio.

### 6.2 Os adaptadores

No Ktor, o stub com corrotinas, sempre com prazo:

```
val resposta = stub
    .withDeadlineAfter(prazo.inWholeMilliseconds, TimeUnit.MILLISECONDS)
    .avisar(avisarRequest {
        tarefaId = tarefa.id.toLong()
        titulo = tarefa.titulo
    })
```

`avisarRequest { }` é uma DSL gerada a partir do `.proto` pelo plugin `kotlin` do protoc: uma lambda com receptor, como as da aula de Kotlin. Uma `StatusException` vira um registro no log e `false`.

No Quarkus, o stub é injetado a partir da configuração:

```
@GrpcClient("avisos")
AvisosServiceBlockingStub stub;
```

```
quarkus.grpc.clients.avisos.host=${AVISOS_HOST:localhost}
quarkus.grpc.clients.avisos.port=${AVISOS_PORTA:50051}
```

O stub é bloqueante porque o recurso já roda numa thread de trabalho.

### 6.3 Gerar os stubs no build

| | Como |
|---|---|
| Ktor | plugin `com.google.protobuf` no `build.gradle.kts`, com `srcDir("../protos")` e os geradores `grpc` (Java) e `grpckt` (Kotlin); o BOM `grpc-bom` alinha a versão de todos os módulos `io.grpc` |
| Quarkus | extensão `quarkus-grpc`; no `pom.xml`, a propriedade `quarkus.grpc.codegen.proto-directory` aponta para `../protos` |

Sem o BOM, o `grpc-kotlin` traz uma versão antiga do `grpc-stub`, e o código gerado não compila. É um exemplo de conflito de dependências que vale reconhecer: uma classe ou método "não encontrado" em código gerado costuma ser versão desalinhada.

📖 Ref. gRPC — Kotlin quick start: https://grpc.io/docs/languages/kotlin/quickstart/

📖 Ref. Quarkus — gRPC clients: https://quarkus.io/guides/grpc-service-consumption

### 6.4 Degradação, conferida

| Serviço Go | Resposta da API | No log |
|---|---|---|
| no ar | `201` em cerca de 0,5 s | as três entregas |
| parado | `201` em cerca de 0,02 s | `não enviado: UNAVAILABLE` |
| lento demais | `201` em cerca de 1 s | `não enviado: DEADLINE_EXCEEDED` |

Sem prazo, um serviço lento prenderia cada requisição HTTP pelo tempo que ele levasse.

### 6.5 Testes

- O cliente contra um servidor gRPC em memória (`grpc-inprocess` no Ktor): resposta rápida, serviço lento além do prazo (o teste confere que volta em menos de 2 s) e serviço fora do ar.
- A rota com um `AvisadorEmMemoria`: criar uma tarefa avisa; um aviso que falha não impede a criação.
- A rubrica pede também um teste de integração do fluxo gRPC de ponta a ponta: a API chamando um servidor de verdade. O `docker compose` do capítulo 7 é o ambiente para isso.

## 7. Containers e `docker compose`

### 7.1 Dockerfiles em dois estágios

O primeiro estágio compila; o segundo leva só o resultado. Ferramentas de build não vão para a imagem de produção.

- **Go:** compila com `golang:1.26`, com `CGO_ENABLED=0` para um binário estático, e copia só o binário para `distroless/static`, uma imagem sem shell. Resultado: cerca de 29 MB.
- **Ktor e Quarkus:** compilam com o JDK 25 e rodam com o JRE 25. Resultado: cerca de 600 MB. O contexto do build é a pasta `exemplos/`, porque os dois precisam de `protos/`.

📖 Ref. Docker — multi-stage builds: https://docs.docker.com/build/building/multi-stage/

### 7.2 Health checks

| Serviço | Health check |
|---|---|
| PostgreSQL | `pg_isready` |
| API Ktor | `GET /health` |
| API Quarkus | `GET /q/health`, da extensão `quarkus-smallrye-health`, que também verifica o banco |
| Go | `avisos saude`: o próprio binário chama o health check gRPC, porque a imagem não tem `curl` |

### 7.3 O `compose.yaml`

O `exemplos/compose.yaml` sobe o banco, o serviço Go e a API Ktor; a API Quarkus fica no perfil `quarkus`. Cada serviço espera os de que depende estarem saudáveis (`depends_on` com `service_healthy`). Dentro da rede do compose, o nome do serviço é o endereço: `avisos:50051`, `banco:5432`.

```
cd exemplos
docker compose up --build
docker compose --profile quarkus up --build
docker compose ps
```

É o critério "Ambiente e testes": `docker compose up` sobe API, serviço Go e banco do zero, sem passos manuais.

📖 Ref. Compose — services: https://docs.docker.com/reference/compose-file/services/

# 8. Implantação

## 8.1 A mesma imagem, configurada pelo ambiente

| Variável | Desenvolvimento | No compose | Em produção |
|---|---|---|---|
| `DB_URL`, `DB_USER`, `DB_PASSWORD` | o banco local | `banco:5432` | a connection string do Neon |
| `AVISOS_ENDERECO` (Ktor), `AVISOS_HOST` (Quarkus) | `localhost` | `avisos` | o endereço interno do serviço Go |
| `PORT` | 8080 | 8080 | a porta que a plataforma define |

Nada de configuração de ambiente no código. Senhas e connection strings são segredos: entram no painel da plataforma, nunca no repositório.

## 8.2 Migração por comando

`ktor-tarefas migrar` aplica as migrações e termina. É o comando que uma plataforma roda antes de a versão nova entrar no ar, para que a aplicação nunca suba com um esquema velho. Rodar na subida da própria aplicação (como o exemplo também faz) funciona, mas mistura duas coisas: uma migração que falha impede a aplicação de subir, e com várias instâncias cada uma tenta migrar ao iniciar. Como comando separado, a migração roda uma vez, e a versão nova só entra no ar se ela der certo.

## 8.3 Do commit à imagem publicada

O `exemplos/implantacao/ci-e-imagens.yml` é um modelo de workflow para o projeto do grupo:

- em todo push e PR: `buf lint`, `go test` e `./gradlew test` (o runner tem Docker, e o Testcontainers funciona sem mudança);

- na `main`, depois dos testes: publica as imagens no GitHub Container Registry, com a etiqueta do commit e `latest`. Voltar a uma versão é apontar para a etiqueta anterior.

📖 Ref. GitHub — Publishing Docker images: https://docs.github.com/en/actions/tutorials/publish-packages/publish-docker-images

### 8.4 Render e Neon, e um limite

O `exemplos/implantacao/render.yaml` é um modelo de Blueprint: a API como web service, o serviço Go como serviço privado, e o banco no Neon.

O plano gratuito do Render tem limites que afetam diretamente este desenho (consultados em 15/10/2026):

| Limite | Consequência |
|---|---|
| serviço privado não tem instância gratuita | o Go num serviço privado é pago |
| web service gratuito envia, mas não recebe, tráfego pela rede privada | o Go como web service gratuito não é alcançável pela API |
| o endereço público não encaminha gRPC | não dá para chamar o Go pela internet |
| o comando antes da implantação só existe em plano pago | no gratuito, a API migra na subida |

Por isso, na entrega, o critério "Sistema no ar" pede a API e o banco gerenciado no ar, e a API precisa continuar respondendo quando o serviço Go não está acessível (o prazo e a degradação do capítulo 6). O serviço Go é avaliado rodando junto da API no `docker compose`. Pô-lo no ar é opcional: o grupo que fizer isso, num plano pago ou noutra plataforma com HTTP/2 de ponta a ponta, registra como fez numa ADR.

📖 Ref. Render — Private network: https://render.com/docs/private-network

📖 Ref. Render — Free instances: https://render.com/docs/free

📖 Ref. Neon — Connect from any application: https://neon.com/docs/connect/connect-from-any-app

## 9. Exercícios e dúvidas frequentes

### Exercícios

1. Escreva a ADR do serviço Go do grupo: a responsabilidade, por que ele é um serviço à parte, e o que aconteceria se fosse parte da API.
2. No exemplo, acrescente um quarto canal (`sms`, 250 ms) e confira, com `go run ./cmd/demo`, que o tempo total não mudou.

3. Com o servidor no ar, chame o serviço com o `grpcurl` e prazo de 100 ms. Quais canais entregam? Por quê, considerando a folga?
4. Acrescente um campo `prioridade` (inteiro) ao `AvisarRequest`, rode `buf lint` e `buf generate`, e confira que a API continua compilando sem mudar nada. O que aconteceria se o novo campo usasse o número 1?
5. Pare o serviço Go e crie uma tarefa pela API. Quanto tempo a resposta leva? E com o serviço lento (aumente a demora do e-mail para 5 s)?
6. Suba tudo com `docker compose up --build` e confira, com `docker compose ps`, que os serviços ficam saudáveis. Pare o `avisos` e veja o que acontece com a API.

### Dúvidas frequentes

**Precisamos escrever o serviço Go do zero?** Sim, com uma responsabilidade do domínio do projeto. O exemplo mostra a estrutura (domínio, adaptador gRPC, contrato) e as práticas (prazo, erros, testes, health check).

**Os stubs Go entram no repositório?** No exemplo, sim (`avisos-go/gen/`). É o costume em Go, e permite compilar sem o Buf. A rubrica pede que sejam gerados, e não escritos à mão.

**O serviço Go pode ter banco próprio?** Pode, se a responsabilidade pedir. O que não ajuda é o serviço Go ler e escrever nas mesmas tabelas da API: os dois deixam de poder mudar de forma independente.

**Quarkus ou Ktor fazem diferença no gRPC?** O contrato e o servidor Go são os mesmos. Muda o lado do cliente: no Ktor, o plugin do Gradle e o stub com corrotinas; no Quarkus, a extensão e o stub injetado.

## Referências

Todas as páginas foram consultadas em outubro de 2026.

Go


- Curso básico de Go (vídeos do professor): https://www.youtube.com/playlist?list=PLTr2C2xXIaIqO7Yr7FHohhgG_OgONpYIx
- Exercícios do curso: https://github.com/classrooms-fmarquesfilho/aprenda-go-com-testes
- Aprenda Go com Testes: https://larien.gitbook.io/aprenda-go-com-testes/
- Aprenda Go com Testes — Concorrência: https://larien.gitbook.io/aprenda-go-com-testes/primeiros-passos-com-go/concorrencia
- Aprenda Go com Testes — Select: https://larien.gitbook.io/aprenda-go-com-testes/primeiros-passos-com-go/select
- Learn Go with Tests — Context: https://quii.gitbook.io/learn-go-with-tests/go-fundamentals/context
- A Tour of Go: https://go.dev/tour/
- Effective Go: https://go.dev/doc/effective_go
- Go — context: https://pkg.go.dev/context

- Go — errors: https://pkg.go.dev/errors
- Go — sync: https://pkg.go.dev/sync

Contrato e gRPC

- Protocol Buffers — proto3: https://protobuf.dev/programming-guides/proto3/
- Buf — lint rules: https://buf.build/docs/lint/rules/
- gRPC — Introduction: https://grpc.io/docs/what-is-grpc/introduction/
- gRPC — Deadlines: https://grpc.io/docs/guides/deadlines/
- gRPC — Status codes: https://grpc.io/docs/guides/status-codes/
- gRPC — Health checking: https://grpc.io/docs/guides/health-checking/
- gRPC — Kotlin quick start: https://grpc.io/docs/languages/kotlin/quickstart/
- Quarkus — gRPC clients: https://quarkus.io/guides/grpc-service-consumption

Containers e implantação

- Docker — multi-stage builds: https://docs.docker.com/build/building/multi-stage/
- Compose — services: https://docs.docker.com/reference/compose-file/services/
- GitHub — Publishing Docker images: https://docs.github.com/en/actions/tutorials/publish-packages/publish-docker-images
- Render — Blueprint YAML: https://render.com/docs/blueprint-spec
- Render — Private network: https://render.com/docs/private-network
- Render — Free instances: https://render.com/docs/free
- Neon — Connect from any application: https://neon.com/docs/connect/connect-from-any-app